INFO4 FISA

Rapport final

Application de gestion de notes

Conception, déploiement sur machines virtuelles et sur conteneurs, évaluation de performance

Réalisé par

TANABENE Abdelhamid

KSOURI Mohamed Walid

Enseignant responsable

M. Mohamed Quafafou

Année universitaire 2026-2027

Rapport final : Application de gestion de notes

Ce rapport présente notre projet d'application de gestion de notes. Nous avons d'abord maquetté l'application sur Figma, puis nous l'avons développée (base PostgreSQL, API Python, client web). Nous l'avons ensuite déployée en architecture 3 tiers, d'abord sur trois machines virtuelles Azure, puis sur trois conteneurs Docker. Enfin, nous avons évalué ses performances avec JMeter.

Récapitulatif des parties réalisées

Partie	Réalisée	Section
Prototype Figma	Oui	1
Conception, analyse et développement de l'application	Oui	2
Déploiement sur les machines virtuelles	Oui	3
Déploiement sur les conteneurs	Oui	4
Évaluation de performance avec JMeter	Oui	5
Rédaction du rapport final	Oui	ce document

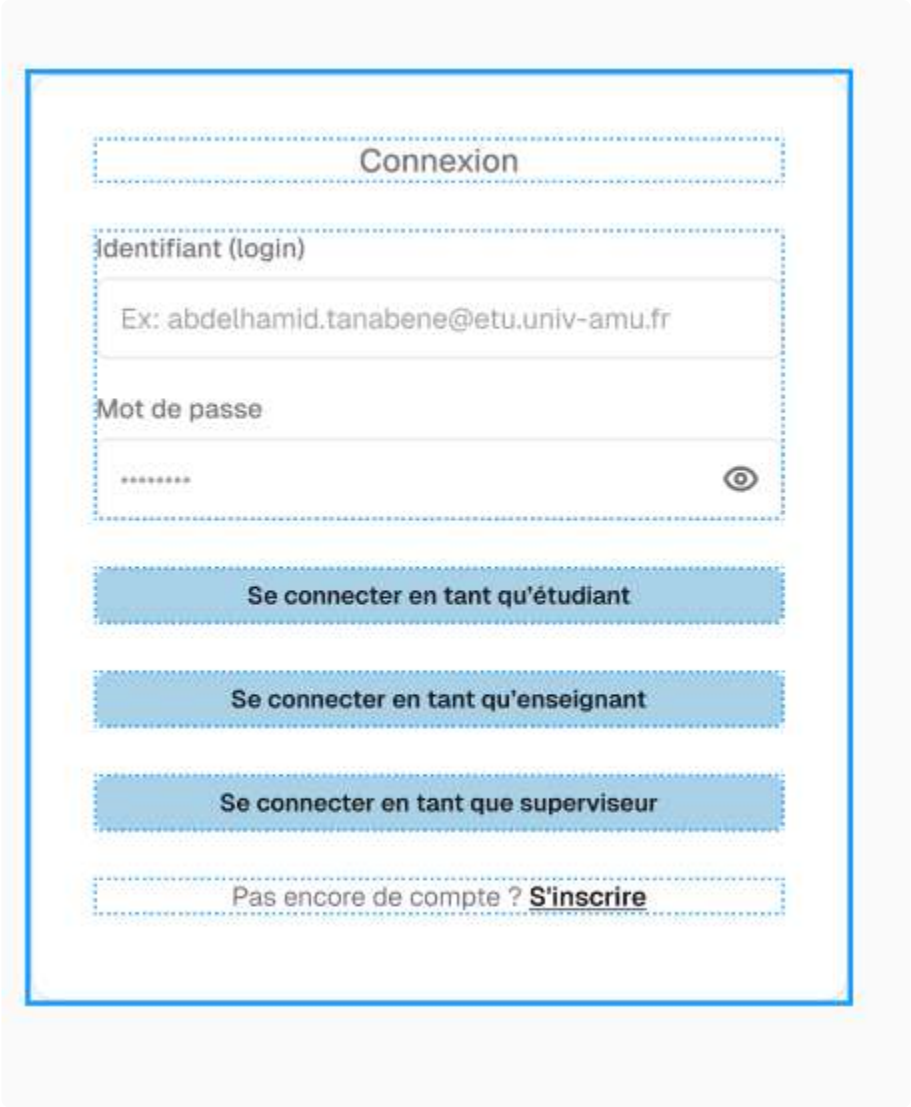
1. Prototype Figma

Avant d'écrire du code, nous avons maquetté les écrans de l'application sur Figma. La maquette nous a servi à fixer les parcours de chaque type d'utilisateur, puis à déduire les routes de l'API dont le client aurait besoin.

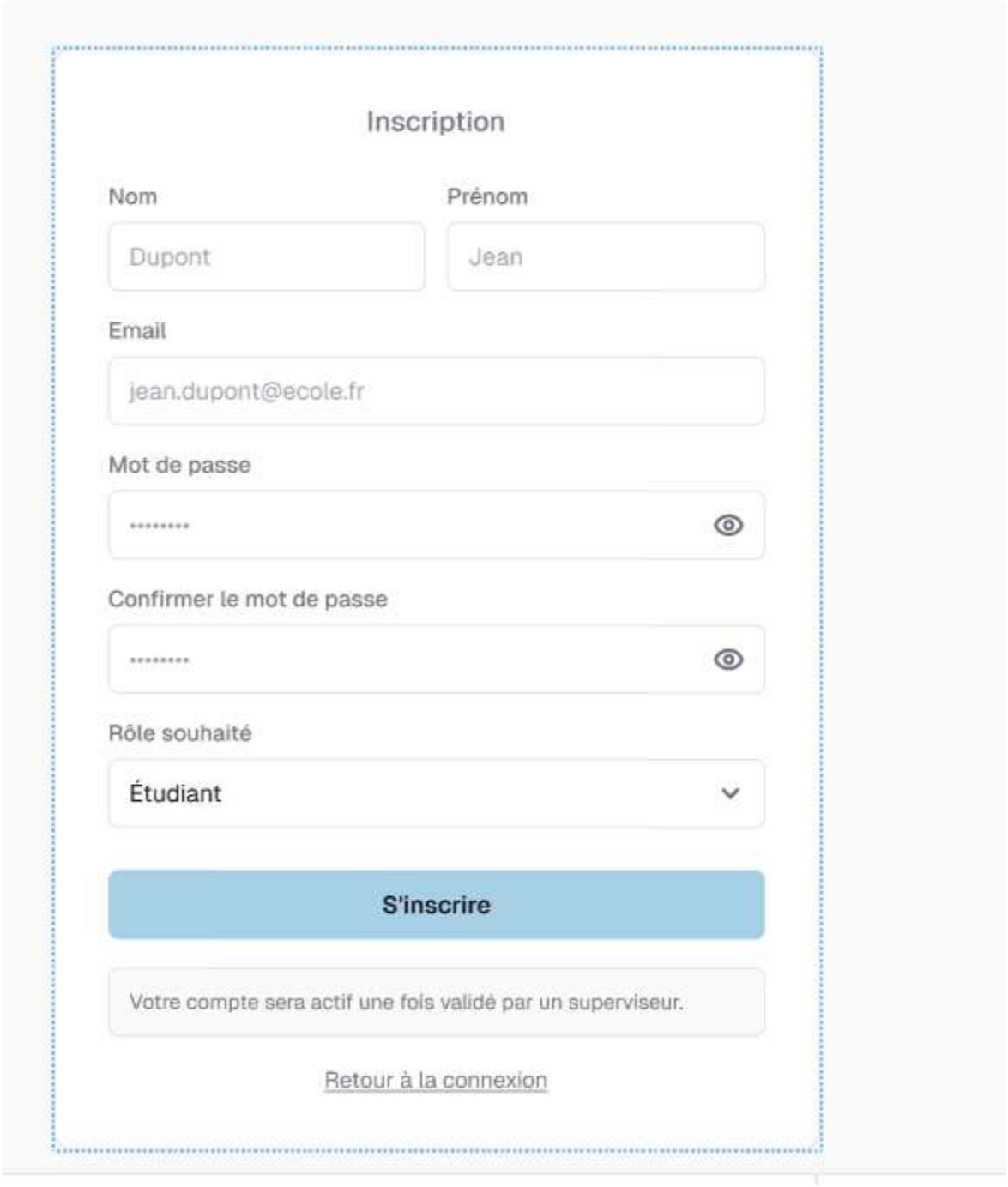
Les écrans maquettés suivent les trois rôles de l'application : étudiant, enseignant et superviseur.

- **Connexion** : identifiant (adresse universitaire), mot de passe, un bouton de connexion par rôle et un lien vers l'inscription.
- **Inscription** : nom, prénom, email, mot de passe et sa confirmation, rôle souhaité. Le compte ne devient actif qu'après validation par un superviseur.
- **Mes notes** (étudiant) : un tableau Cours, Note, Date. L'en-tête rappelle le rôle et le nom de l'utilisateur, avec un bouton de déconnexion.

- **Notes d'un cours** (enseignant) et **demandes d'inscription** (superviseur), qui correspondent aux routes `/api/courses/{id}/grades` et `/api/registration-requests`.



Maquette 1 : connexion, avec un bouton par rôle



Maquette 2 : inscription, avec choix du rôle et validation par un superviseur



Maquette 3 : notes de l'étudiant

La page de connexion de l'application déployée (figure 7, section 3) reprend les champs et le lien « S'inscrire » de la maquette, mais avec un seul bouton « Se connecter » au lieu d'un bouton par rôle.

2. Conception, analyse et développement

L'application permet à des utilisateurs de se connecter, de consulter des cours et de gérer des notes. Elle distingue trois rôles : l'étudiant consulte ses notes, l'enseignant gère les notes de ses cours, et le superviseur valide les demandes d'inscription des nouveaux comptes.

Choix techniques

Couche	Technologie	Rôle
Présentation	Client web statique, servi par Nginx	Pages et appels à l'API
Métier	API Python (ASGI), servie par uvicorn sur le port 8000	Authentification, règles métier, accès aux données
Données	PostgreSQL 16	Stockage

Le dépôt est organisé par couche : `client/`, `métier/`, `données/` (scripts SQL), `deploiement/` et `docs/`, avec un `requirements.txt` pour les dépendances Python. Cette séparation nous a permis de déployer chaque couche indépendamment, sur une VM puis dans un conteneur.

Modèle de données

La base `notes_db` contient quatre tables, créées par les scripts du dossier `données/` :

Table	Contenu
`users`	Les comptes (identifiant, mot de passe chiffré, rôle)
`courses`	Les cours
`grades`	Les notes, chacune reliée à un étudiant et à un cours
`registration_requests`	Les demandes d'inscription en attente de validation

Routes de l'API

Les principales routes de l'API sont les suivantes :

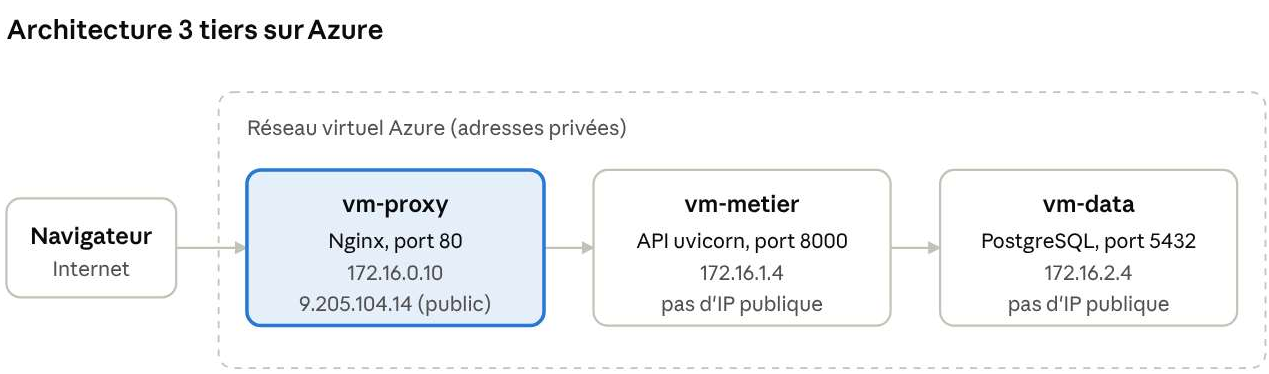
Méthode	Route	Usage
GET	`/api/health`	Vérifier que l'API répond
POST	`/api/auth/login`	Se connecter
GET	`/api/students`	Lister les étudiants
GET	`/api/students/{id}/grades`	Notes d'un étudiant
GET	`/api/courses`	Lister les cours
GET	`/api/courses/{id}/grades`	Notes d'un cours
GET	`/api/registration-requests`	Demandes d'inscription en attente

Les paramètres de connexion à la base (hôte, port, nom, utilisateur, mot de passe) ne sont pas écrits dans le code. Ils sont lus dans des variables d'environnement, ce qui permet de changer d'environnement (VM ou Docker) sans toucher au code.

3. Déploiement sur les machines virtuelles

Nous avons déployé chaque couche sur sa propre VM Azure (Ubuntu 24.04, taille Standard B1s : 1 vCPU, 1 Gio de RAM), chacune dans un sous-réseau du même réseau virtuel. Seul le proxy est joignable depuis Internet.

Machine	Rôle	IP privée	IP publique
vm-proxy	Reverse proxy Nginx (port 80)	172.16.0.10	9.205.104.14
vm-metier	API Python, uvicorn (port 8000)	172.16.1.4	aucune
vm-data	PostgreSQL 16 (port 5432)	172.16.2.4	aucune



Le métier et les données n'ont pas d'IP publique ; PostgreSQL n'accepte que 172.16.1.4.

Schéma de l'architecture 3 tiers sur Azure

Le navigateur ne communique qu'avec le proxy. Le proxy transmet les appels à l'API, qui est la seule à interroger la base.

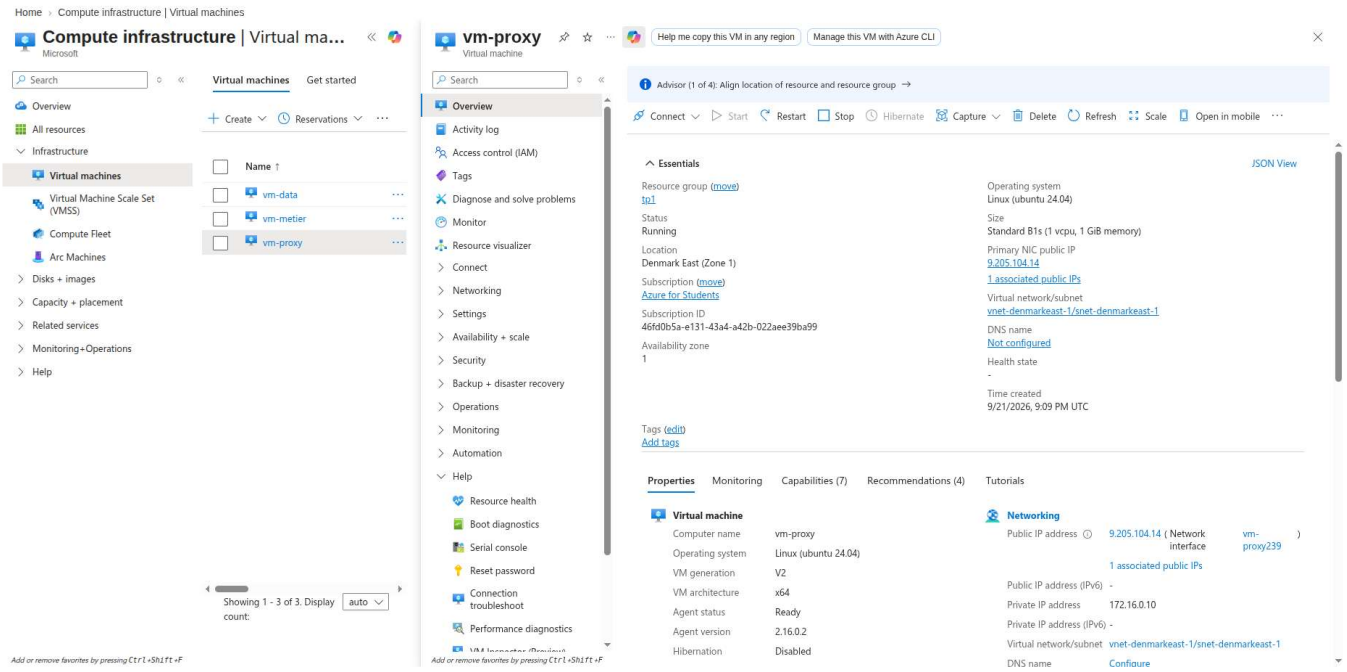


Figure 1 : vm-proxy dans le portail Azure (IP privée 172.16.0.10, IP publique 9.205.104.14)

Couche données (vm-data)

Nous avons installé PostgreSQL 16, créé la base `notes_db` et l'utilisateur `notes_user`, puis exécuté les scripts SQL. La figure 2 montre les quatre tables créées.

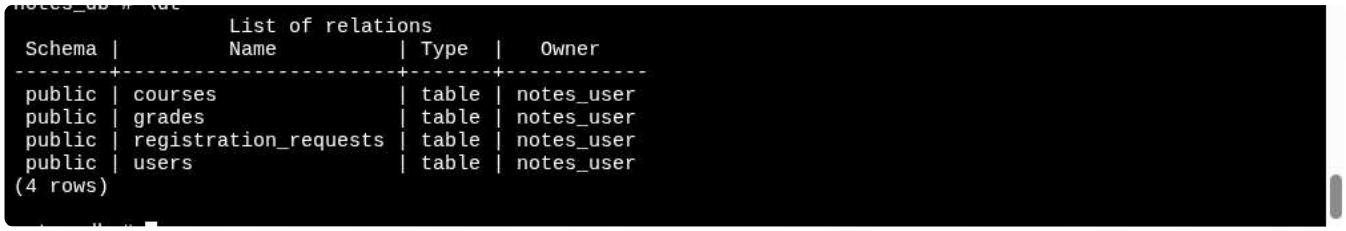


Figure 2 : les quatre tables de notes_db

PostgreSQL écoute sur toutes les interfaces (`listen_addresses = '*'`). C'est `pg_hba.conf` qui limite l'accès : une seule règle autorise `notes_user` sur `notes_db`, depuis 172.16.1.4/32 (la VM métier), en scram-sha-256. Mettre `listen_addresses = '172.16.2.4'` aurait été plus strict.

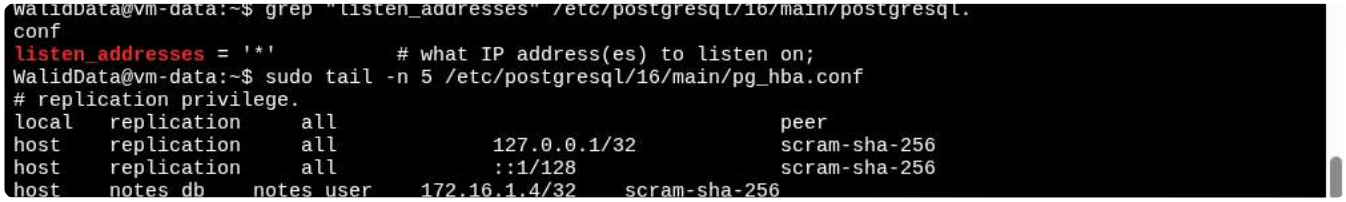


Figure 3 : listen_addresses et la règle d'accès de pg_hba.conf

Couche métier (vm-metier)

Nous avons installé Python 3 et un environnement virtuel, puis lancé l'API :

```
source venv/bin/activate
uvicorn métier.app.main:app --host 0.0.0.0 --port 8000
```

Les paramètres de connexion sont dans un fichier `.env` (`DB_HOST=172.16.2.4`, `DB_PORT=5432`, `DB_NAME=notes_db`, `DB_USER=notes_user`, mot de passe masqué). La VM n'ayant pas d'IP publique, le port 8000 n'est joignable que depuis le réseau privé. Les journaux montrent que toutes les requêtes viennent de 172.16.0.10, le proxy, et répondent 200 OK.

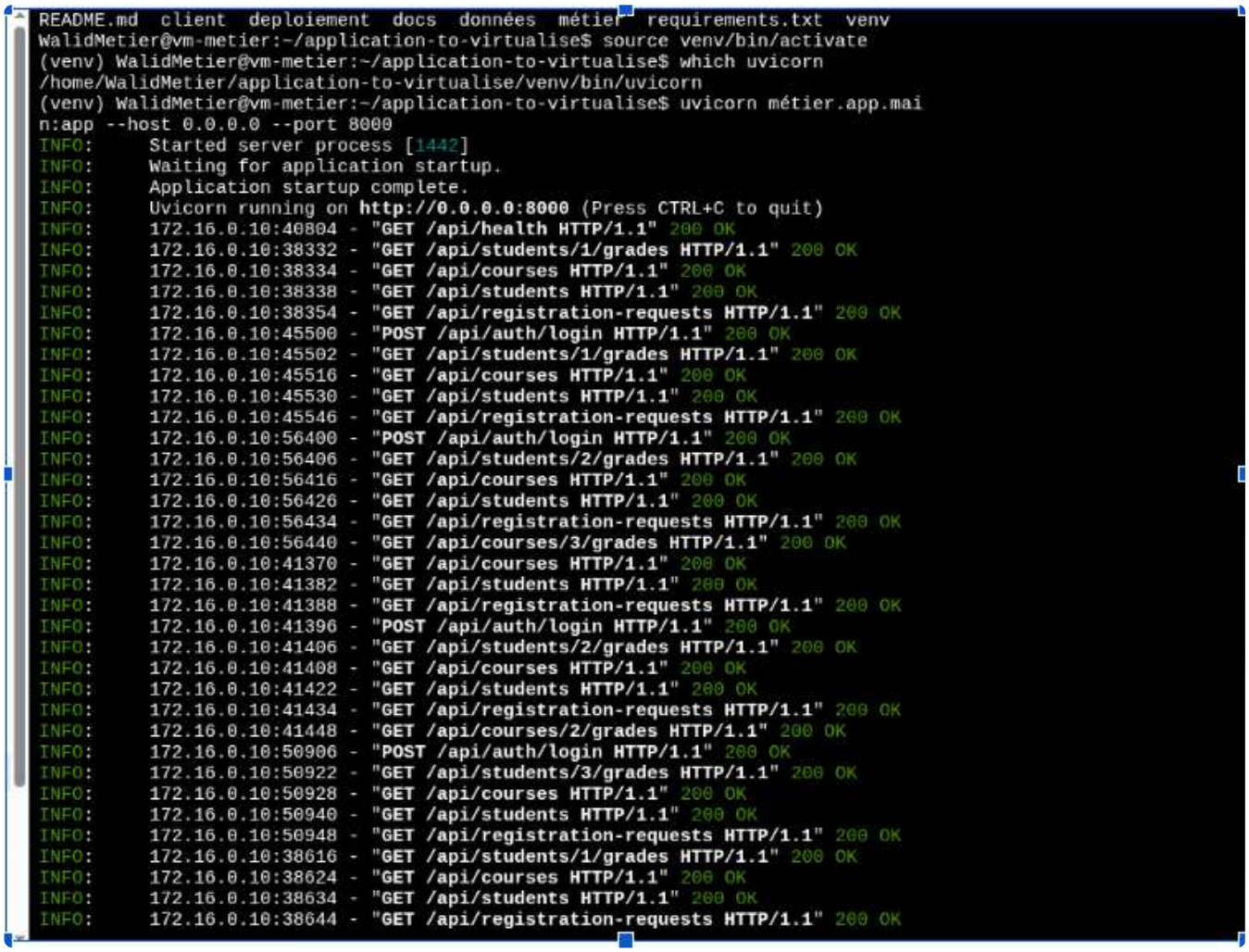


Figure 4 : uvicorn sur le port 8000, requêtes reçues du proxy (172.16.0.10)

Couche proxy (vm-proxy)

Nginx sert les fichiers du client et transmet les appels `/api/` à la VM métier :

```
location /api/ {
    proxy_pass http://172.16.1.4:8000;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}
```

Le groupe de sécurité réseau du proxy autorise HTTP (80) et SSH (22) en entrée. Celui de vm-metier ne garde que les règles par défaut d'Azure : trafic interne autorisé, tout le reste refusé. Limite : la règle SSH était ouverte à toute source, il aurait fallu la limiter à notre IP.

Virtual network / subnet
vnet-denmarkeast-1 / snet-denmarkeast-1

Public IP address
9.205.104.14

Private IP address
172.16.0.10

Admin security rules
0 (Configure)

Application security groups
0 (Configure)

Network security group
vm-proxy-nsg

Accelerated networking
Disabled

Effective security rules
0

Rules Collapse all

Network security group vm-proxy-nsg (attached to networkInterface: vm-proxy239)
Impacts 0 subnets, 1 network interfaces

Create port rule

Search rules | Source == all | Destination == all | Protocol == all | Action == all | Port == all

Prio... ↑	Name	Port	Protocol	Source	Destination	A
Inbound port rules (5)						
300	HTTP	80	TCP	Any	Any	
320	SSH	22	TCP	Any	Any	
65000	AllowVnetInBound	Any	Any	VirtualNetwork	VirtualNetwork	
65001	AllowAzureLoadBalancerInB...	Any	Any	AzureLoadBalancer	Any	
65500	DenyAllInBound	Any	Any	Any	Any	
Outbound port rules (3)						

Figure 5 : règles entrantes de vm-proxy (HTTP 80 et SSH 22)

Rules Collapse all

Network security group vm-metier-nsg (attached to networkInterface: vm-metier806)
Impacts 0 subnets, 1 network interfaces

Create port rule

Search rules | Source == all | Destination == all | Protocol == all | Action == all | Port == all

Prio... ↑	Name	Port	Protocol	Source	Destination	A
Inbound port rules (3)						
65000	AllowVnetInBound	Any	Any	VirtualNetwork	VirtualNetwork	
65001	AllowAzureLoadBalancerInB...	Any	Any	AzureLoadBalancer	Any	
65500	DenyAllInBound	Any	Any	Any	Any	
Outbound port rules (3)						

Figure 6 : règles entrantes de vm-metier (aucun accès depuis Internet)

Validation

Depuis un navigateur, l'IP publique du proxy affiche la page de connexion, et la connexion d'un utilisateur fonctionne. La requête traverse les trois couches : Nginx, puis l'API sur 172.16.1.4:8000, puis PostgreSQL sur 172.16.2.4:5432.

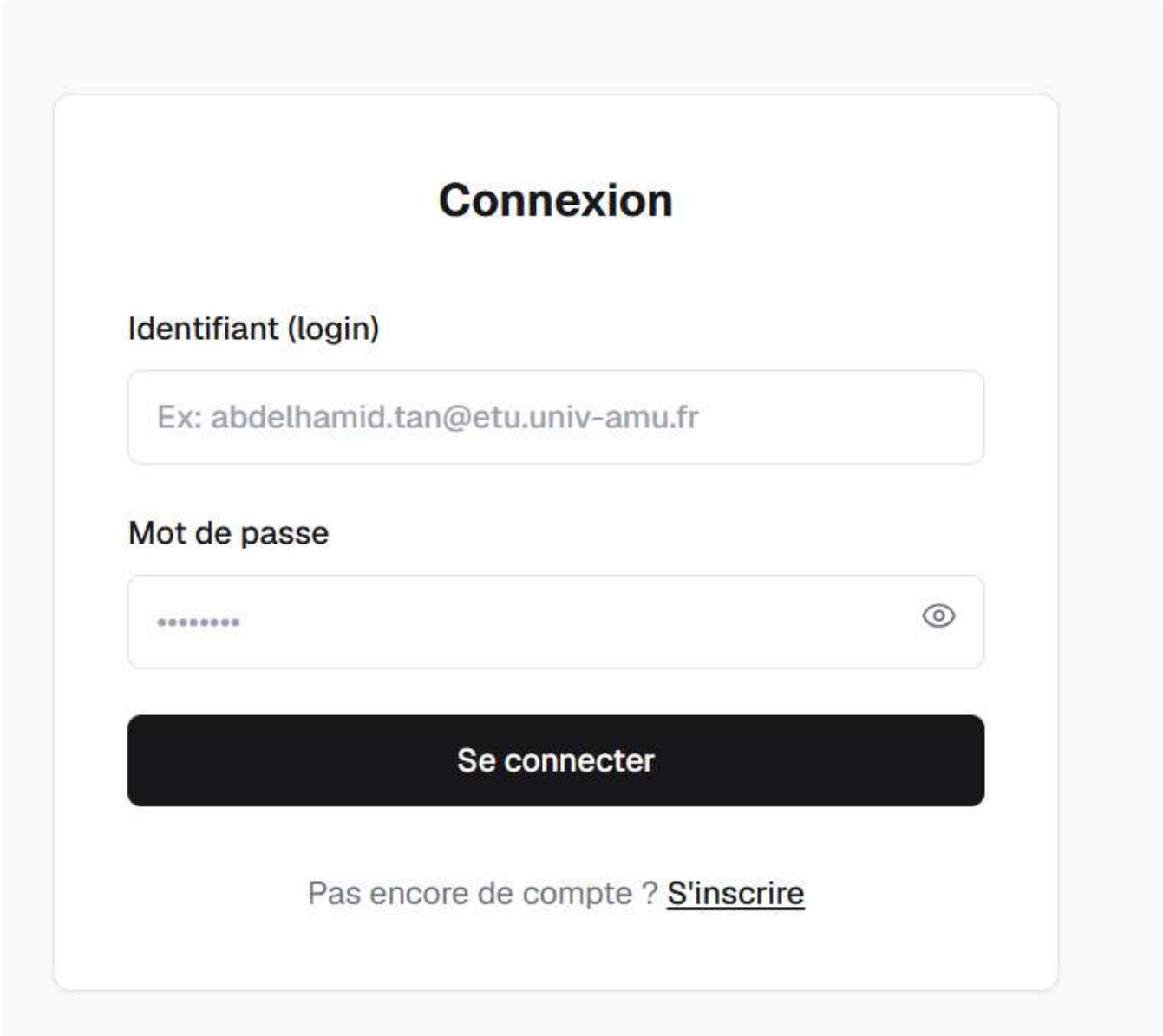


Figure 7 : page de connexion servie par le proxy

4. Déploiement sur les conteneurs

Nous avons repris la même application avec une couche par conteneur Docker, les trois décrites dans un seul `compose.yaml` et lancées sur une seule VM (vm-proxy, 1 Gio de RAM).

Service	Image	Port	Particularité
`frontend`	`notes-frontend` (Nginx 1.27.5)	80, publié sur l'hôte	Sert le client et transmet `/api/` au backend
`backend`	`notes-backend` (`python:3.12-slim`)	8000, interne (`expose`)	Démarre quand la base est « healthy »
`DB`	`postgres:16-alpine`	5432, interne	Volume `postgres_data`, scripts d'initialisation, healthcheck `pg_isready`

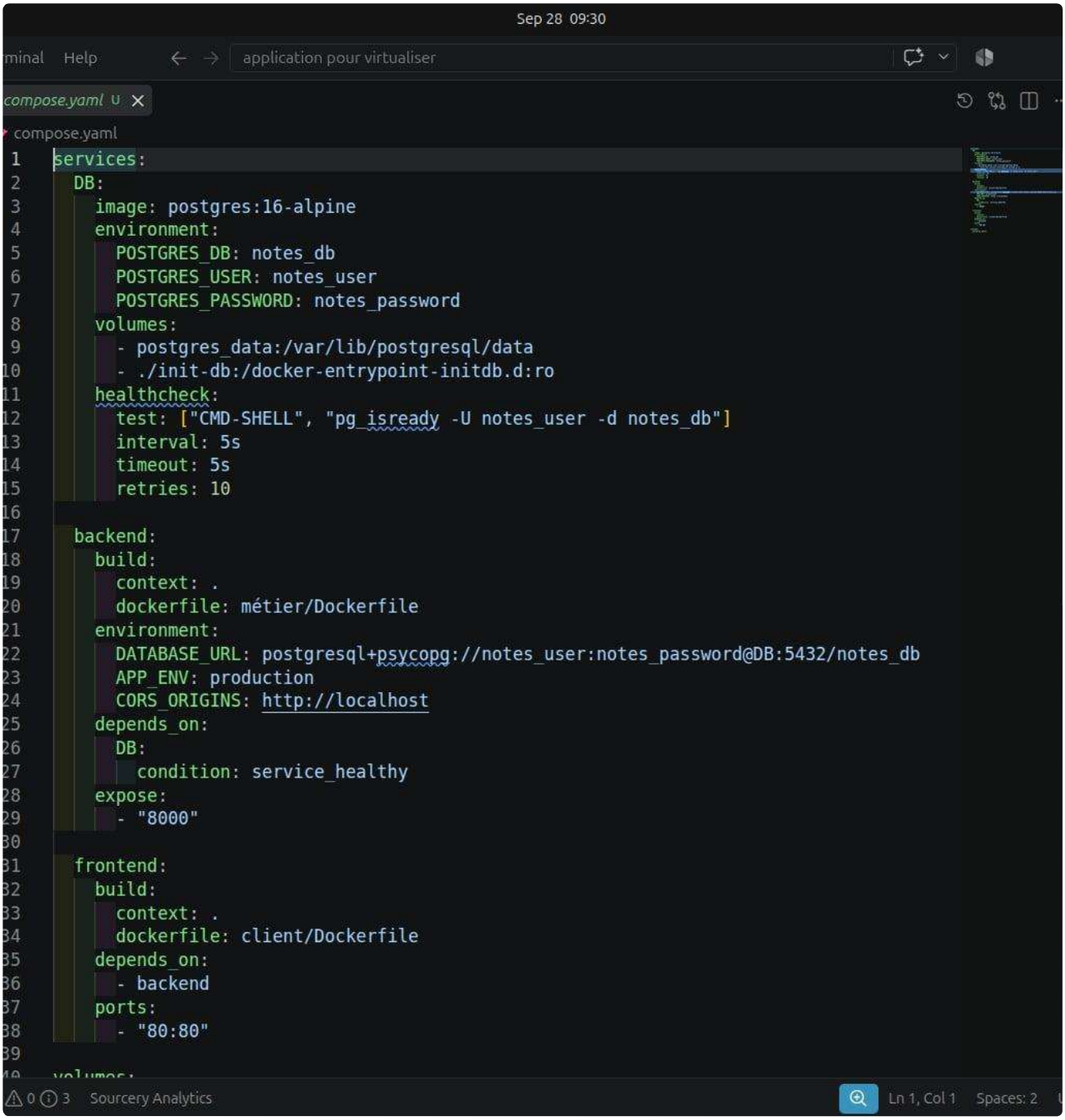


Figure 8 : fichier compose.yaml

Couche données

Le volume nommé garde les données si le conteneur est supprimé. Le dossier `init-db` est monté en lecture seule dans `/docker-entrypoint-initdb.d`, ce qui crée les tables au premier démarrage. Aucun port de la base n'est publié vers l'hôte.

Couche métier

Le Dockerfile part de `python:3.12-slim`. Il copie `requirements.txt` et installe les dépendances avant de copier le code : tant que les dépendances ne changent pas, Docker réutilise cette couche en cache. L'API se connecte à la base par le nom du service, `DB:5432`, que Docker traduit en adresse IP. Un `.dockerignore` exclut `.git`, l'environnement virtuel, les caches Python, les données locales et le fichier `.env`.



Figure 9 : Dockerfile de la couche métier

Lancement et vérification

`docker compose up -d --build` démarre les trois conteneurs dans l'ordre. Depuis l'hôte, `curl http://localhost/api/health` renvoie `{"status":"ok"}` : la requête passe par Nginx et arrive à l'API.

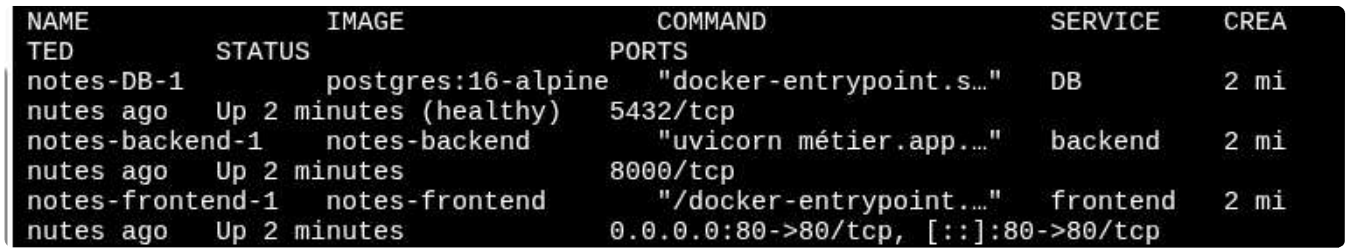


Figure 10 : les trois conteneurs en marche, DB à l'état healthy

Depuis	Vers	Résultat
Hôte	proxy, port 80	joignable
Hôte	métier, port 8000	non joignable (port non publié)
Hôte	données, port 5432	non joignable (port non publié)
Conteneur proxy	métier, port 8000	joignable
Conteneur proxy	données	joignable, alors qu'il ne devrait pas
Conteneur métier	données, port 5432	joignable

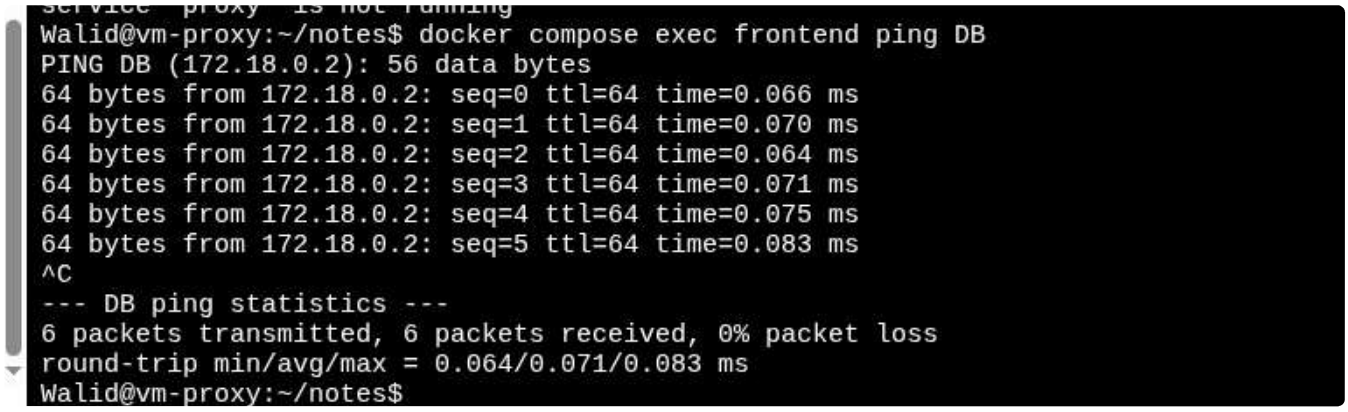


Figure 11 : le conteneur proxy joint la base (0 % de perte)

Lors de ce test, nous avons remarqué un problème : le conteneur proxy arrive à joindre la base (`ping DB` sans perte), alors qu'il ne devrait pas. Comme nous n'avions déclaré aucun réseau, Docker a placé les trois services sur le même réseau par défaut. Pour corriger cela, il faut déclarer deux réseaux, le backend étant le seul à être sur les deux :

```
services:
  DB:
    networks: [back]
  backend:
    networks: [front, back]
  frontend:
    networks: [front]

networks:
  front:
  back:
    internal: true
```

Avec cette configuration, le frontend ne peut plus résoudre le nom DB, et seul le backend accède à la base.

Ressources consommées

Conteneur	Mémoire (Mio)	Image (Mo)
frontend	6,73	73,7
backend	62,93	307
DB	22,25	420
Total	**91,91**	**800,7**

```
Walid@vm-proxy:~/notes$ docker stats --no-stream
CONTAINER ID   NAME               CPU %     MEM USAGE / LIMIT     MEM %     NET
I/O           BLOCK I/O         PIDS
abd9b5109d14   notes-frontend-1   0.00%     6.727MiB / 836.4MiB   0.80%     22.3
kB / 69.7kB   15.9MB / 12.3kB   2
12155ed509e7   notes-backend-1    0.12%     62.93MiB / 836.4MiB   7.52%     17.4
kB / 15.9kB   81.6MB / 885kB    6
f18480301630   notes-DB-1         0.00%     22.25MiB / 836.4MiB   2.66%     14.6
kB / 10.1kB   78.8MB / 623kB    7
Walid@vm-proxy:~/notes$
```

Figure 12 : mémoire consommée (docker stats)

```
Walid@vm-proxy:~/notes$ docker system df -v
Images space usage:

REPOSITORY       TAG         IMAGE ID       CREATED          SIZE      SHARED SI
ZE   UNIQUE SIZE   CONTAINERS
notes-backend    latest      cf34731e08d0   30 minutes ago   307MB     2B
     306.7MB       1
notes-frontend   latest      37ae2bda8094   30 minutes ago   73.7MB    2B
     73.65MB       1
postgres         16-alpine   721873c34ceb   10 days ago      420MB     0B
     419.7MB       1

Containers space usage:

CONTAINER ID   IMAGE                COMMAND                  LOCAL VOLUMES   SIZ
E      CREATED          STATUS                     NAMES
abd9b5109d14   d2c8607c4335         "/docker-entrypoint.…"   0               81.
9kB    26 minutes ago   Up 26 minutes              notes-frontend-1
12155ed509e7   af970d49c87c         "uvicorn métier.app.…"   0               1.0
2MB    26 minutes ago   Up 26 minutes              notes-backend-1
f18480301630   postgres:16-alpine   "docker-entrypoint.s…"   1               20.
5kB    26 minutes ago   Up 26 minutes (healthy)    notes-DB-1
```

Figure 13 : espace disque des images et des conteneurs (docker system df -v)

5. Évaluation de performance avec JMeter

Nous avons évalué les performances du déploiement Docker avec Apache JMeter, en simulant plusieurs utilisateurs qui se connectent puis consultent leurs notes.

Environnement de test

Nous avons déployé les trois conteneurs sur un PC portable avec `docker compose up --build -d`. Le dossier `charge/` (plan de test et comptes) est monté en lecture seule dans le backend. JMeter 5.6.3 tourne en ligne de commande (`-n`) et vise `localhost:80` : chaque requête passe par le proxy Nginx, comme pour un utilisateur normal.

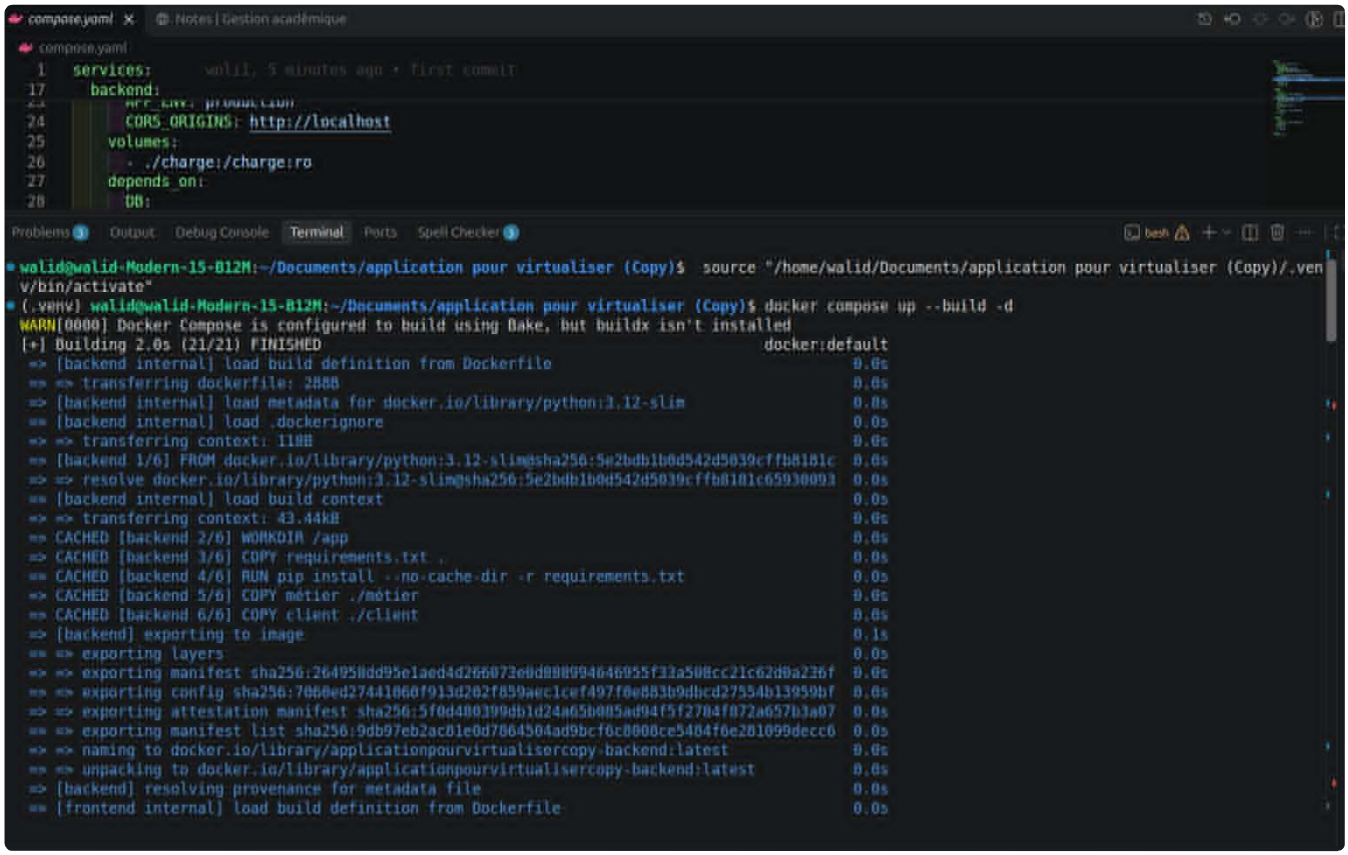


Figure 14 : déploiement local des trois conteneurs (backend et frontend démarrés, DB healthy)

Plan de test

- **Scénario** (`charge/plan.jmx`) : chaque utilisateur virtuel se connecte (`POST /api/auth/login`) avec un compte lu dans `comptes.csv`, puis consulte ses notes (`GET /api/students/{id}/grades`).
- **Paramètres** : montée en charge sur 30 s, durée de 90 s, paliers de 1, 5, 10, 25, 50 et 100 utilisateurs. Chaque palier produit son fichier de résultats `.jtl`.
- **Analyse** : JMeter génère un tableau de bord HTML à partir d'un fichier `.jtl` (`jmeter -g <fichier.jtl> -o <dossier>`).

```
for users in 1 5 10 25 50 100; do
  jmeter -n -t charge/plan.jmx -l "charge/resultats/test-${users}.jtl" \
    -Jhost=localhost -Jport=80 -Jusers="$users" \
    -Jramp_up=30 -Jduration=90 -Jaccounts="$PWD/charge/comptes.csv"
done
```

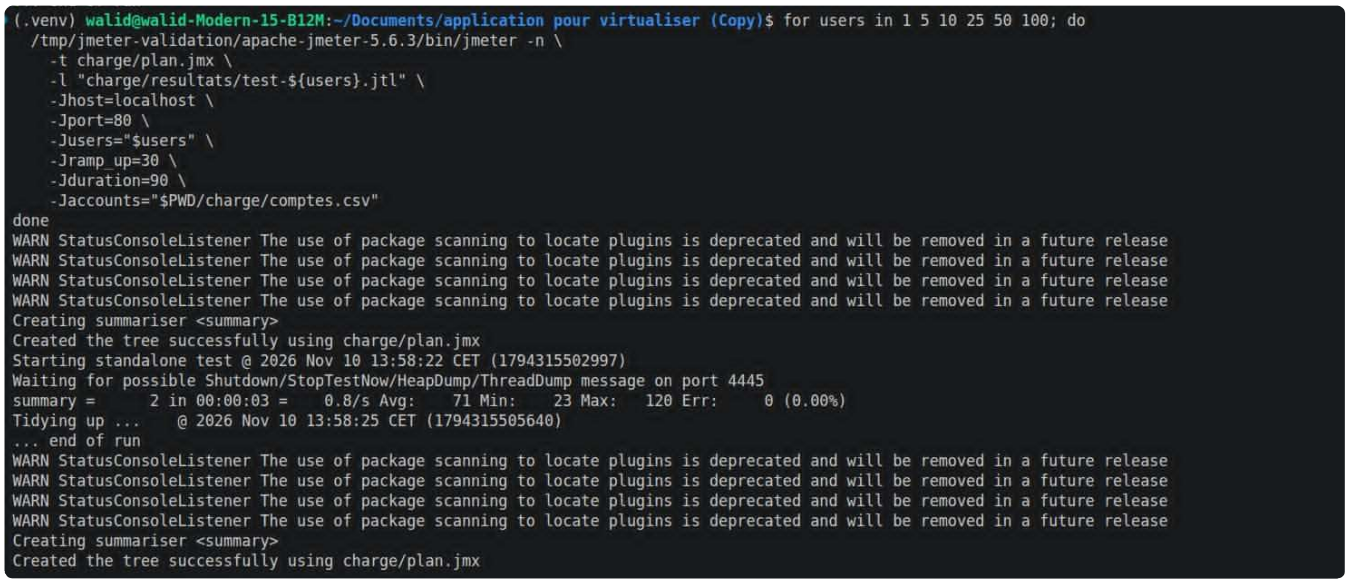


Figure 15 : boucle de tests sur les six paliers

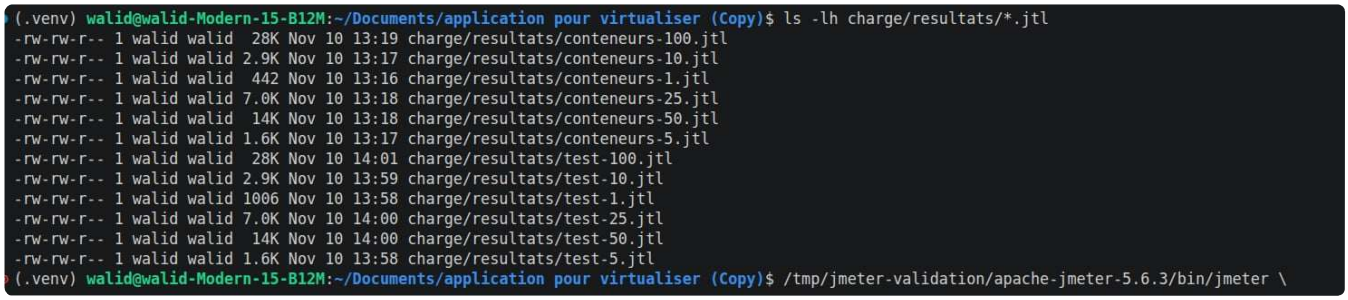


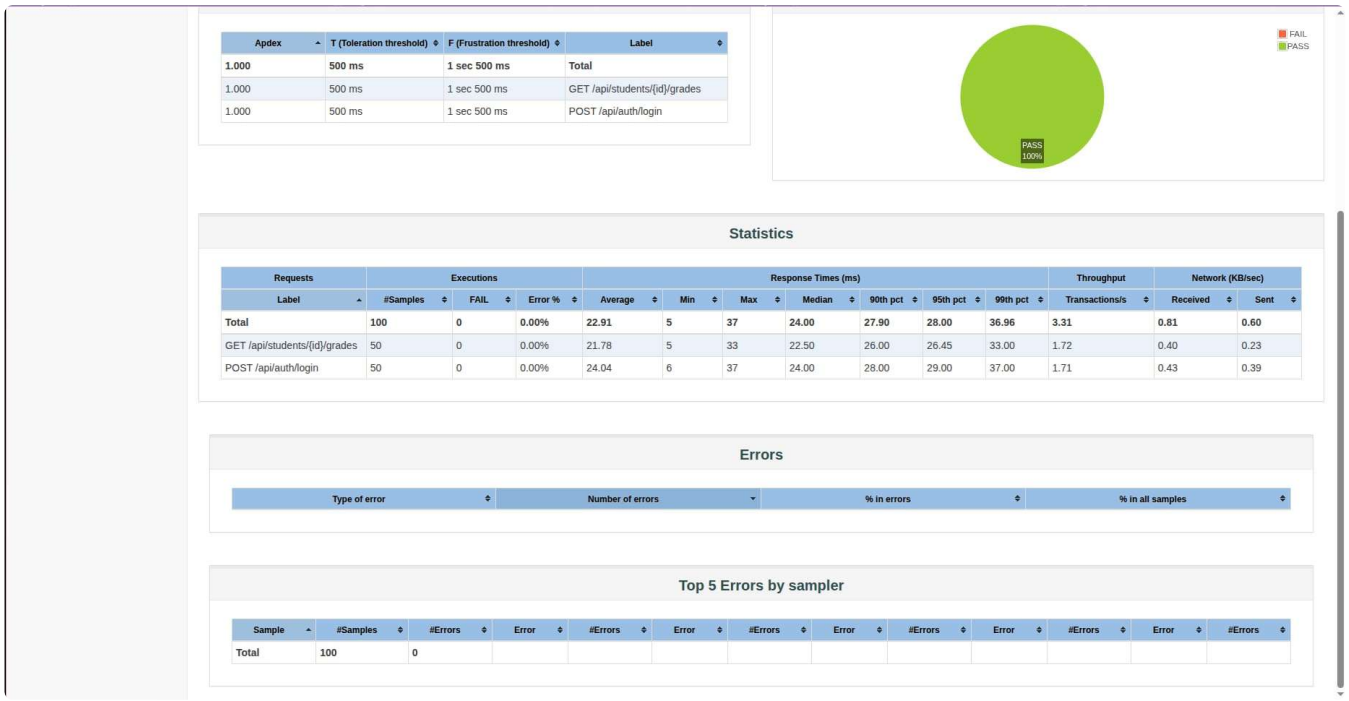
Figure 16 : fichiers de résultats produits pour chaque palier

Résultats

Résultats du palier à 50 utilisateurs (100 requêtes), tirés du rapport HTML :

Requête	Échantillons	Moyenne (ms)	Médiane (ms)	90e centile (ms)	99e centile (ms)	Max (ms)	Débit (req/s)	Erreurs
Total	100	22,91	24,00	27,90	36,96	37	3,31	0 %
`POST /api/auth/login`	50	24,04	24,00	28,00	37,00	37	1,71	0 %
`GET /api/students/{id}/grades`	50	21,78	22,50	26,00	33,00	33	1,72	0 %

L'Apdex vaut 1,000 pour les deux requêtes : toutes les réponses sont sous le seuil de tolérance de 500 ms.



Apdex	T (Toleration threshold)	F (Frustration threshold)	Label
1.000	500 ms	1 sec 500 ms	Total
1.000	500 ms	1 sec 500 ms	GET /api/students/{id}/grades
1.000	500 ms	1 sec 500 ms	POST /api/auth/login

Statistics

Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent
Total	100	0	0.00%	22.91	5	37	24.00	27.90	28.00	36.96	3.31	0.81	0.60
GET /api/students/{id}/grades	50	0	0.00%	21.78	5	33	22.50	26.00	26.45	33.00	1.72	0.40	0.23
POST /api/auth/login	50	0	0.00%	24.04	6	37	24.00	28.00	29.00	37.00	1.71	0.43	0.39

Errors

Type of error	Number of errors	% in errors	% in all samples

Top 5 Errors by sampler

Sample	#Samples	#Errors	Error	#Errors	Error	#Errors	Error	#Errors	Error	#Errors	Error	#Errors
Total	100	0										

Figure 17 : rapport HTML JMeter du test à 50 utilisateurs (Apdex, statistiques, erreurs)

Le premier test à 1 utilisateur (2 requêtes) donne 203 ms de moyenne (de 75 à 331 ms). Relancé dans la boucle, le même palier descend à 71 ms. Les toutes premières requêtes sont plus lentes, sans doute parce que l'API et la base viennent de démarrer.

```
(.venv) walid@walid-Modern-15-B12M:~/Documents/application pour virtualiser (Copy)$ /tmp/jmeter-validation/apache-jmeter-5.6.3/bin/jmeter -n \
  -t charge/plan.jmx \
  -l charge/resultats/test-1.jtl \
  -Jhost=localhost \
  -Jport=80 \
  -Jusers=1 \
  -Jramp_up=30 \
  -Jduration=90 \
  -Jaccounts="$PWD/charge/comptes.csv"
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
Creating summariser <summary>
Created the tree successfully using charge/plan.jmx
Starting standalone test @ 2026 Nov 10 13:57:19 CET (1794315439043)
Waiting for possible Shutdown/StopTestNow/HeapDump/ThreadDump message on port 4445
summary =      2 in 00:00:03 =    0.7/s Avg:   203 Min:    75 Max:   331 Err:     0 (0.00%)
Tidying up ...    @ 2026 Nov 10 13:57:21 CET (1794315441846)
... end of run
```

Figure 18 : premier test à 1 utilisateur

Interprétation

À 50 utilisateurs, tous les temps restent sous 40 ms : l'application est donc loin d'être saturée. Avec une montée en charge sur 30 s, les utilisateurs n'arrivent pas tous en même temps (environ 1,7 par seconde), et chacun ne fait qu'une connexion et une consultation.

Ces tests ont une limite : JMeter et les conteneurs tournaient sur le même PC et se partageaient le processeur. Pour trouver la charge maximale supportée, il faudrait raccourcir la montée en charge ou répéter le scénario en boucle pendant toute la durée du test.

6. Comparaison et conclusion

Les trois conteneurs utilisent 91,91 Mio de RAM au total, contre 3 Gio réservés pour les trois VM. Ils tiennent largement dans une seule de nos VM.

Critère	3 VM (Azure B1s)	3 conteneurs
Mémoire	3 Gio alloués (1 Gio par VM)	91,91 Mio utilisés
Disque	Un disque système complet par VM	800,7 Mo d'images
Démarrage	Démarrage d'un OS complet, de l'ordre de la minute	Quelques secondes
Isolation	Forte : un noyau par VM, séparation par l'hyperviseur et les sous-réseaux	Plus faible : noyau partagé, séparation réseau à configurer

Une VM émule une machine complète, avec son noyau et son système. Un conteneur n'est qu'un processus isolé qui utilise le noyau de l'hôte : l'image ne contient que l'application et ses dépendances. L'isolation repose sur le noyau (namespaces et cgroups).

En contrepartie, l'isolation est plus faible. Une faille du noyau ou une panne de l'hôte touche les trois couches à la fois. La séparation réseau dépend aussi entièrement de la configuration : avec les VM, chaque couche avait son sous-réseau et ses règles ; avec Docker, oublier de déclarer les réseaux a suffi pour que le proxy joigne la base.

Conclusion

Les deux déploiements font tourner la même application sans modifier son code, car les couches sont séparées et la configuration passe par des variables d'environnement. Docker consomme beaucoup moins de ressources et se met en place plus vite, alors que les VM isolent mieux les couches par défaut. Pour améliorer notre déploiement, il faudrait séparer les réseaux Docker, sortir les mots de passe du fichier `compose.yaml` et limiter l'accès SSH à notre adresse IP.